

Communication Complexity: Communication Lower Bounds For Multi-Party Graph Traversal

Ryan Batubara

rbatubara@ucsd.edu

Ivy Hawks

m1hawks@ucsd.edu

Darren Ho

rbatabura@ucsd.edu

Ciro Zhang

ciz001@ucsd.edu

Mentor: Dr. Shachar Lovett

slovett@ucsd.edu

Motivation

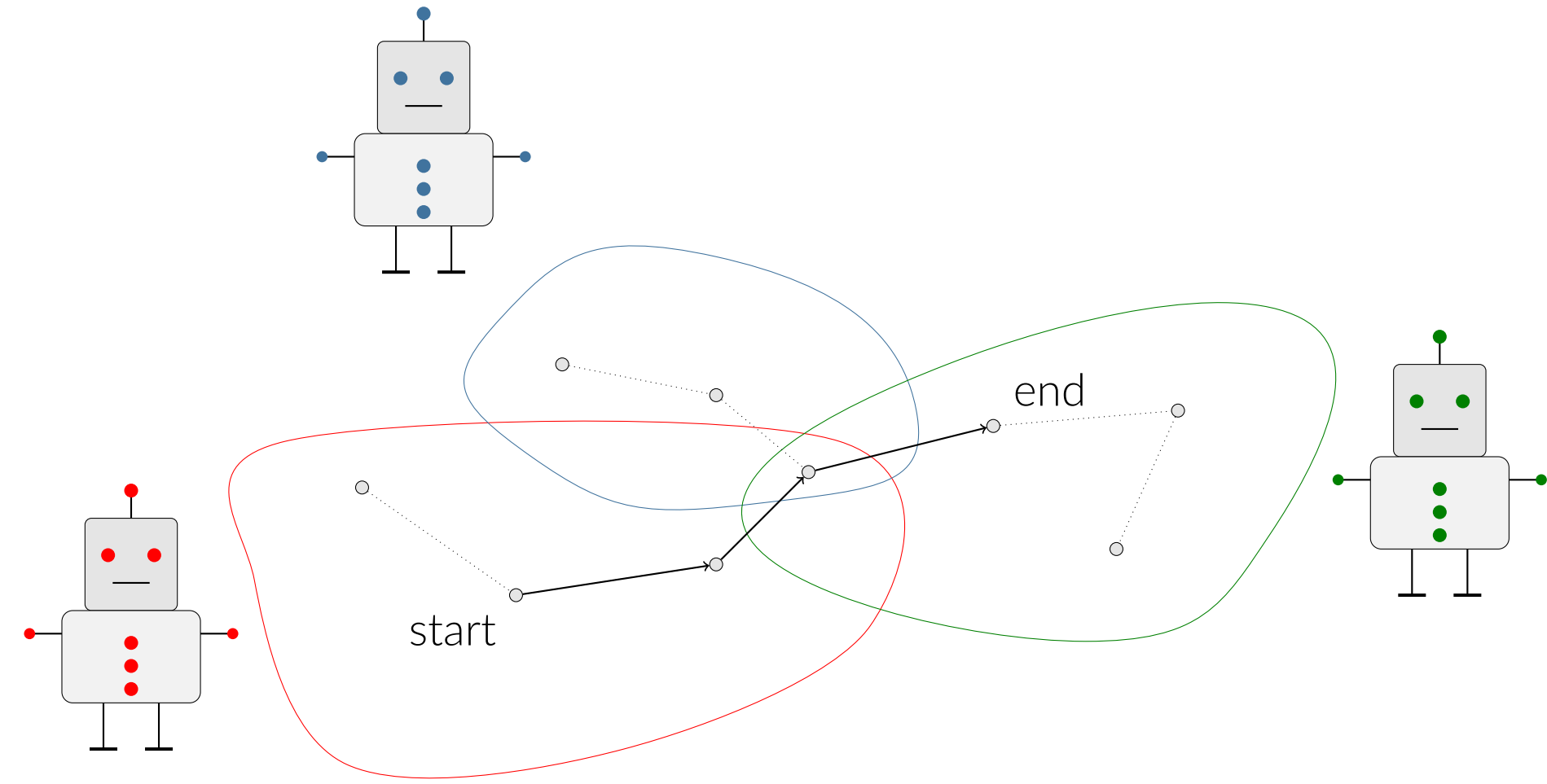


Figure 1. Path in Agent Owned Graphs

In multi-agentic learning systems, there is a strong need for multiple agents to be able to coordinate while learning a problem. However, it is also important for those agents to be able to communicate the minimum amount of information possible to optimize the computational cost of the overall process. It is therefore natural to consider questions in Communication Complexity, which asks, for any function,

How much communication is fundamentally necessary?

We consider the learning space as a graph, which each agent will need to understand properties during training to optimize training parameters.

Two Party Communication Complexity

Communication Complexity studies the minimum number of bits that two parties must exchange to compute a function when the input is distributed.

Instead of asking *How fast is the algorithm?*,

We ask *How much communication is needed within the algorithm?*

In this model, there are two players, traditionally called **Alice** and **Bob**.

- Alice holds input x
- Bob holds input y

Neither of them knows the inputs of the other person, so they must talk back and forth until they know enough to compute some function $f(x, y)$.

In the worst case, Alice just sends all n bits. This is called **maximally hard**, as after sending her entire input, Bob can simply compute the result.

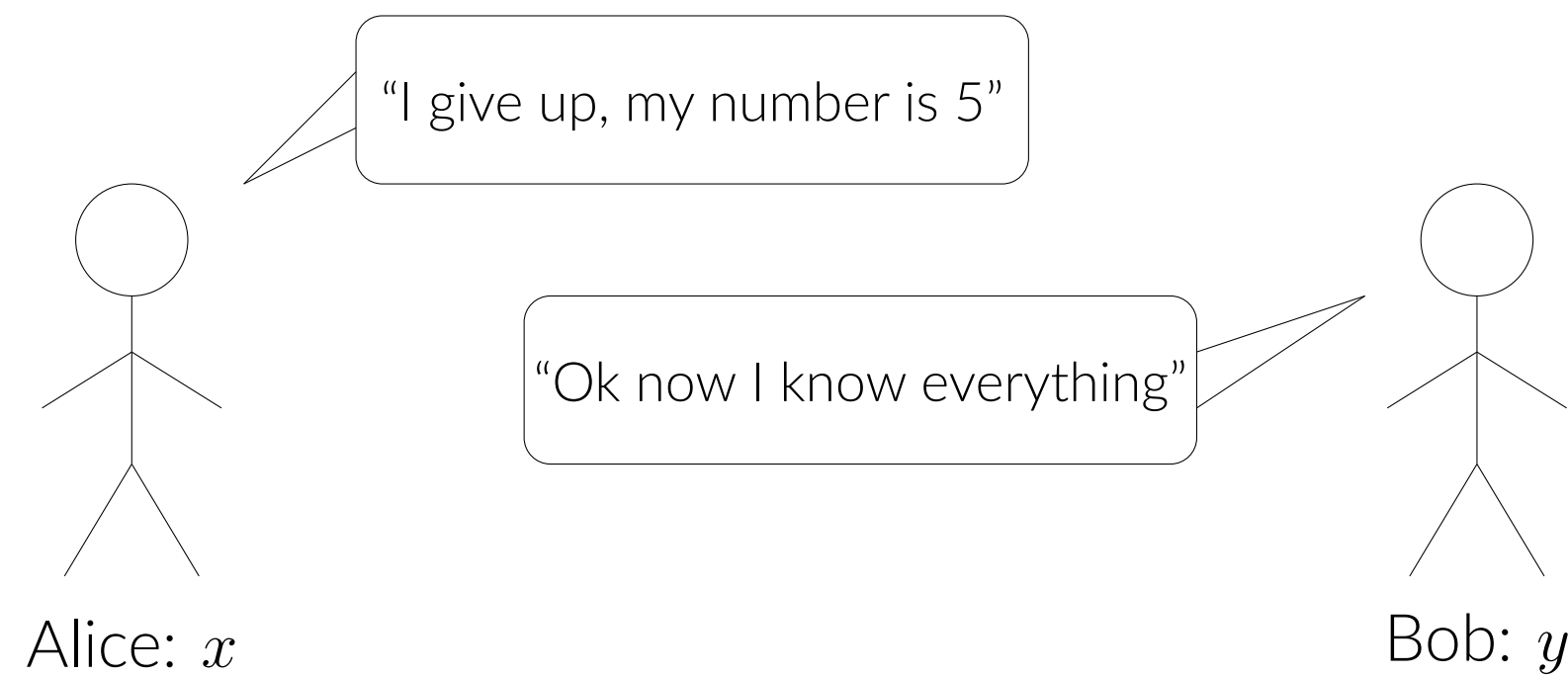


Figure 2. Alice and Bob in the maximally hard case

Equality is Maximally Hard

Equality Problem: Do Alice and Bob have the same n -bit string?

$$EQ(x, y) = \begin{cases} 1 & x = y \\ 0 & x \neq y \end{cases}$$

In Figure 3, we represent EQ as a communication matrix:

- Rows = Alice's input x
- Columns = Bob's input y
- Blue dots mark where $EQ(x, y) = 1$

In Communication Complexity, Alice send bits to Bob to eliminate parts of this matrix, and vice-versa. The regions eliminated are rectangles, so a bound on a function can be proven by finding the minimum number of bits needed to reduce to rectangles containing only 1s or 0s.

But in Equality, each blue dot is isolated, so in order to cover all 2^n diagonal 1's, the protocol needs at least 2^n rectangles. It turns out this implies that Equality requires at least n bits of communication.

Thus, **Equality is maximally hard**.

The Traversal Problem

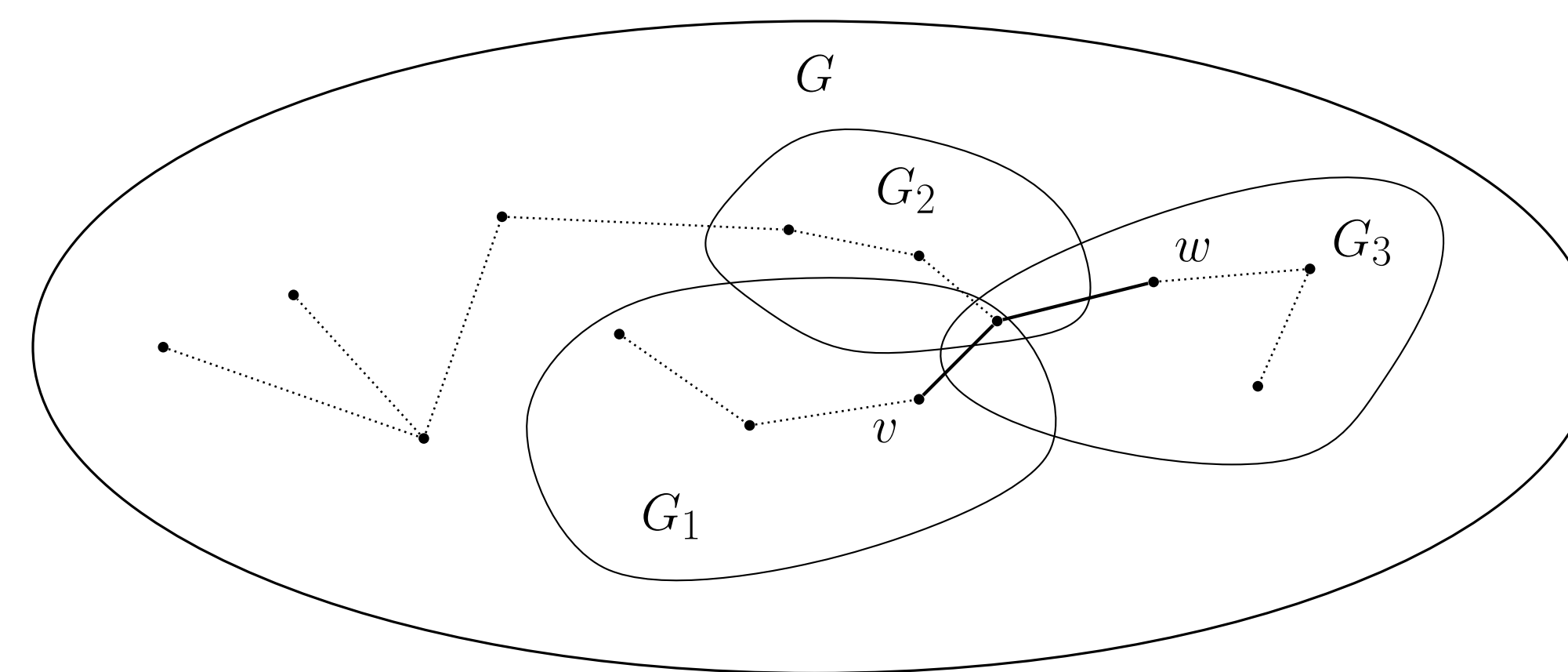


Figure 4. Traversal example where a valid path from v to w is shown in bold.

In this section, we define a simpler version of the problem from our motivation.

Consider a large graph G , known by both players. Inside G , each agent "owns" part of the graph. We call these subgraphs G_1 and G_2 . Neither player knows the other players subgraph.

Given a starting vertex v and a target vertex w , how much information do the player need to share about their subgraphs to determine whether there is a path from v to w , which can only use vertices and edges in the players subgraphs?

Embedding Equality into Traversal Equality Gadget Chain

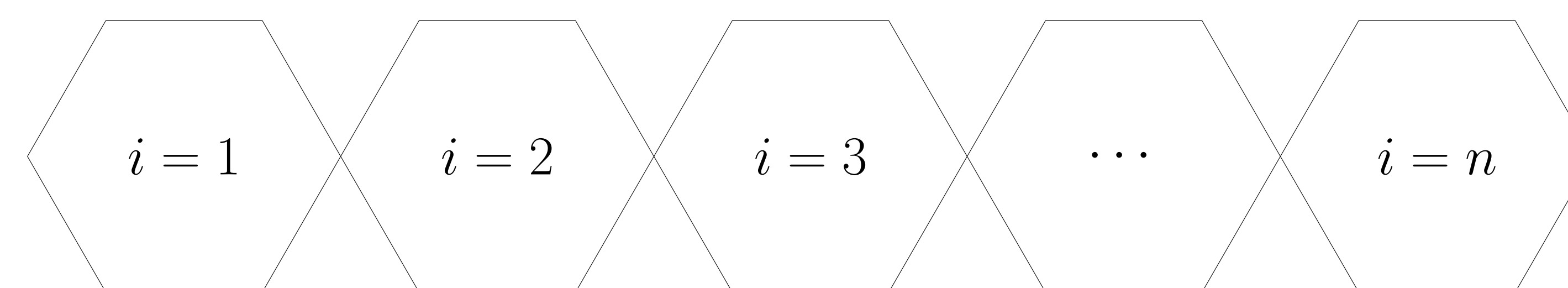


Figure 6. Chain of Equality Gadgets

We can string a set of these graphs along in a chain, one for every index i , so that traversing from one end to the other is only possible if $x_i = y_i$ for every index, which means that it is only possible if $x = y$.

Using this, we can embed Equality into the Traversal problem. Solving this case efficiently would mean checking if $x = y$ efficiently, so any algorithm that solves the Traversal problem will also solve Equality.

From this we can conclude that **Traversal is also deterministically maximally hard**.

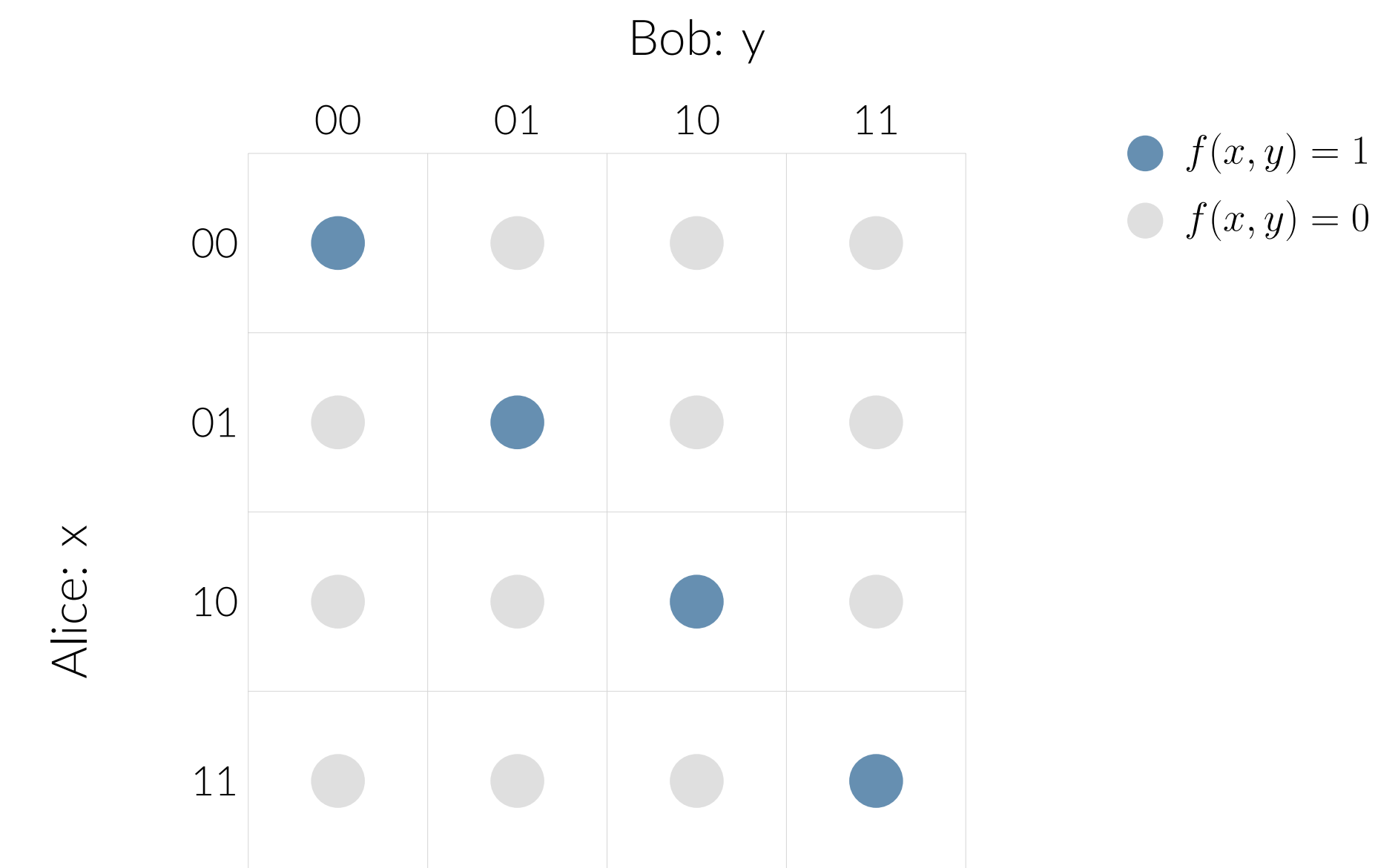


Figure 3. Communication matrix for Equality

The Equality Gadget

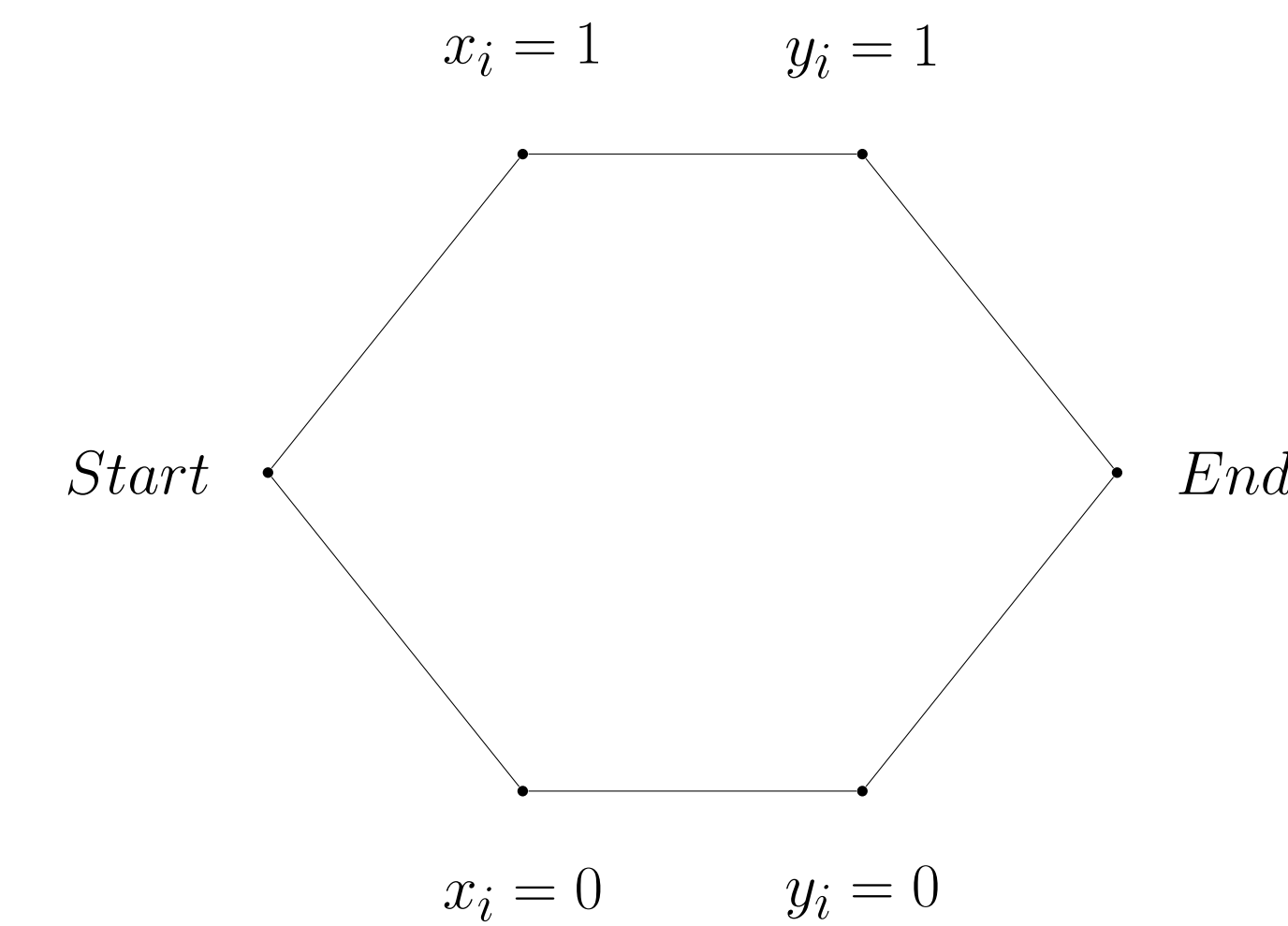


Figure 5. Gadget for proving equality Embedding

Consider Figure 5 as one part of a whole graph. All players own the start and end vertices. Let x_i be the i th bit of Alice's input, and y_i be the i th bit of Bob's input. Then:

1. The top branch corresponds to $x_i = y_i = 1$,
2. The bottom branch corresponds to $x_i = y_i = 0$.

And thus, traversal is only possible if Alice's x_i is equal to Bob's y_i .

Introducing Randomness

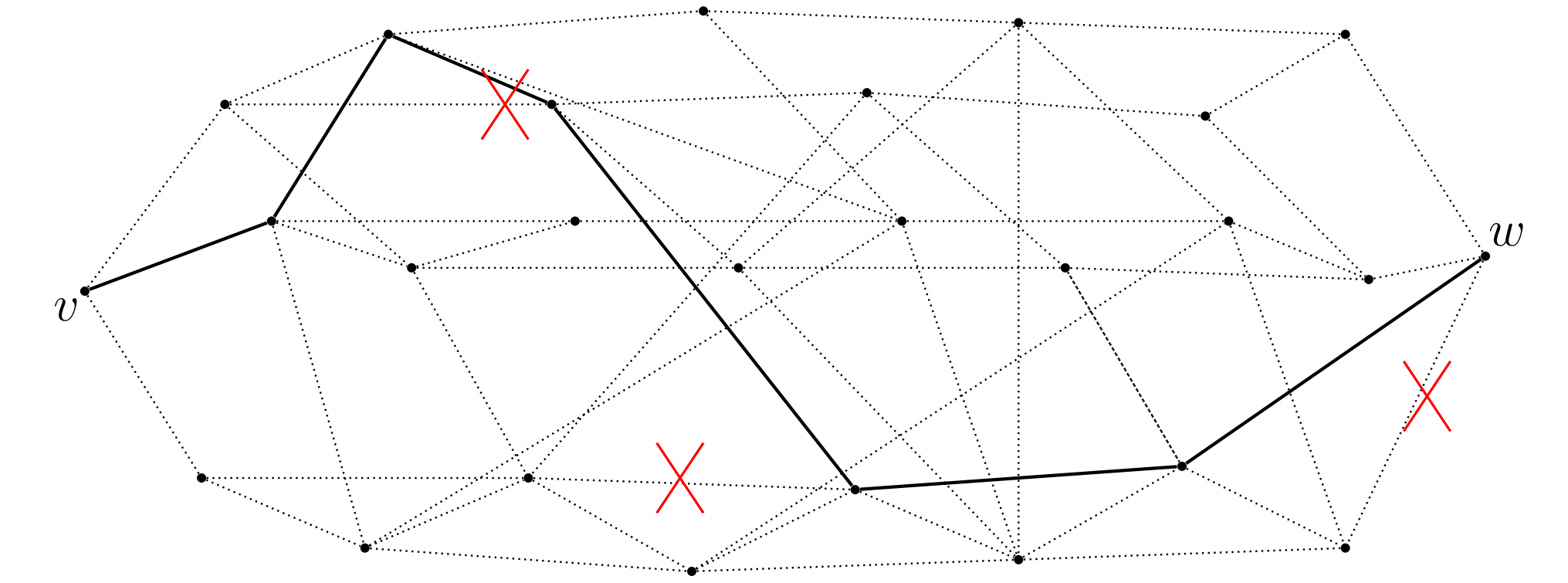


Figure 7. Graph with ignored Edges

Deterministically we have seen that Traversal is **maximally hard**. There is no asymptotically better way to solve this problem than for Alice to send her entire input to Bob. However, we might also consider adding randomness and allowing error. For instance, if there is a path, it is unlikely that it will use any randomly selected edge, so we might consider randomly ignoring some.

Set Disjointness

To find a lower bound for a random algorithm we can no longer rely on embedding Equality, as random algorithms for equality can be bounded as low as $\Theta(1)$. Thus, we turn to Set Disjointness:

$$DISJ(x, y) = \begin{cases} 1 & x \cap y \neq \emptyset, \\ 0 & x \cap y = \emptyset. \end{cases}$$

Set Disjointness gives each player some number of elements from a set, and asks if the players have any elements in common. If so, the protocol outputs 0, and 1 otherwise. Set Disjointness is usefull when studying random algorithms since it has been shown that it requires asymptotically **maximal random communication**.

Disjoint Gadget and Embedding

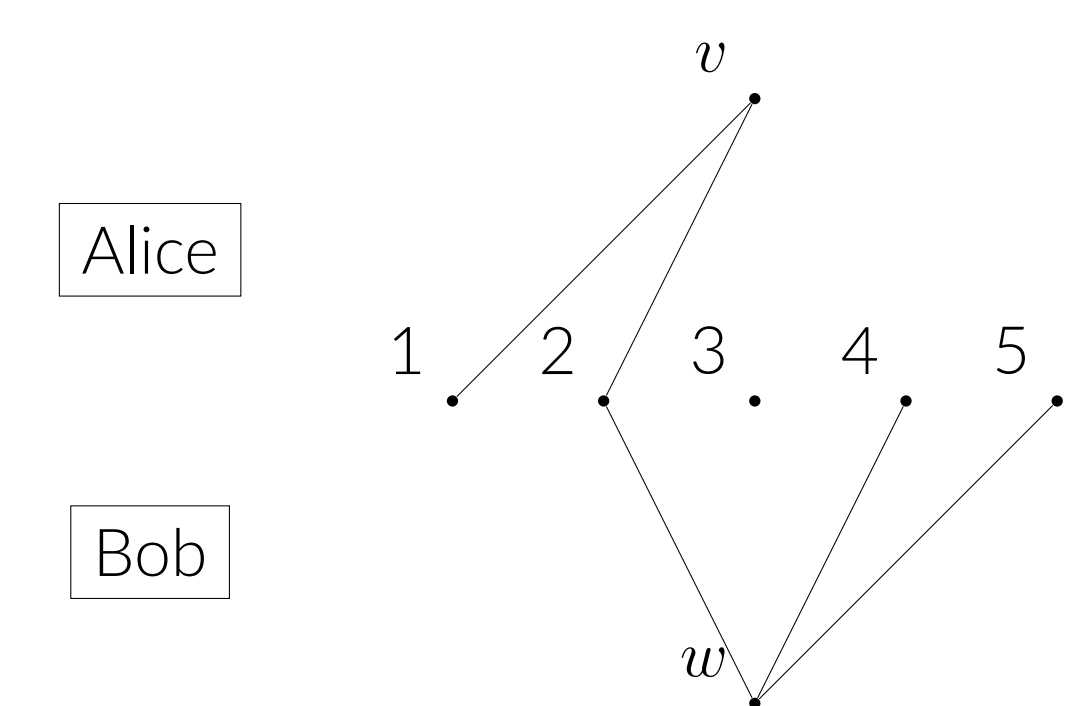


Figure 8. Disjointness Gadget. Here, Alice has $\{1, 2\}$, and Bob has $\{2, 4, 5\}$. There is a path from v to w through 2.

This gadget shows that Set Disjointness can also be embedded into Traversal. We let Alice own edges between v and i , if i is in x , and likewise for Bob. Therefore there is a path between v and w iff $x \cap y \neq \emptyset$.

Multi-Party

In addition, we have found that these results generalize to three or more players as well. Specifically, we show that in this case the problem is still maximally hard in both the Deterministic and Random case.